

SOFTWARE REQUIREMENTS SPECIFICATION

1 Introduction

1.1 Purpose

This document describes the requirements for the Blockchain-Based Company Audit Integrity System, a decentralized application designed to securely store and verify audit reports using blockchain technology.

The system ensures that audit documents cannot be altered without detection by storing their cryptographic hashes on a blockchain.

Traditional audit systems rely on centralized databases which are vulnerable to manipulation and lack transparent timestamping. Blockchain provides an immutable ledger that enhances audit integrity and trust between stakeholders.

1.2 Scope

The system will:

- Allow auditors to upload audit reports
- Generate a SHA-256 hash of the document
- Store the hash on the Ethereum blockchain
- Record timestamp and auditor identity
- Allow verification of audit authenticity
- Provide role-based access (Auditor, Company, Regulator)
- The system will NOT store actual documents on-chain to reduce cost and protect confidentiality.

1.3 Definitions

Term	Definition
Hash	Cryptographic fingerprint of a document
Smart Contract	Self-executing code deployed on blockchain
Auditor	Authorized entity submitting audit reports
Regulator	Authority verifying audit authenticity
IPFS	Decentralized file storage system
Ethereum	Blockchain network used

2 Overall Description

2.1 Product Perspective

The system consists of:

- Frontend (React Web App)
- Smart Contract (Solidity)
- Blockchain Network (Ethereum / Polygon)
- Optional IPFS storage -> well do this if we have time

2.2 System Architecture

Architecture Flow:

- I. User uploads audit document
- II. System generates SHA-256 hash
- III. Hash stored in smart contract
- IV. Blockchain records timestamp
- V. Anyone can verify document integrity

2.3 User Classes

1. Auditor
 - Submit audit reports
 - View submitted reports
 - Update audit status
2. Company
 - View their audit records
 - Verify document authenticity
3. Regulator
 - Verify audit authenticity
 - View audit history

3 Functional Requirements

3.1 Audit Submission

- FR-1: The system shall allow auditors to upload audit reports.
FR-2: The system shall generate a SHA-256 hash of the uploaded document.
FR-3: The system shall store the hash on the blockchain.
FR-4: The system shall record timestamp automatically.
FR-5: The system shall associate audit with company ID.

3.2 Audit Verification

- FR-6: The system shall allow users to upload a document for verification.
FR-7: The system shall generate a hash of the uploaded document.
FR-8: The system shall compare hash with blockchain record.
FR-9: The system shall display verification result (Valid / Tampered).

3.3 Role-Based Access Control

- FR-10: Only authorized auditors can submit audits.
FR-11: Companies can only view their own audit records.
FR-12: Regulators can view all audit records.

3.4 Audit Record Retrieval

- FR-13: The system shall allow retrieval of audit by ID.
FR-14: The system shall display:
 - Auditor address
 - Company ID
 - Document hash
 - Timestamp
 - Status

4 Non-Functional Requirements

4.1 Security

All transactions must require wallet signature.
Smart contract must prevent unauthorized access.
Document contents must not be stored on-chain.

4.2 Performance

Transaction confirmation time ≤ 30 seconds (testnet).
Hash generation ≤ 2 seconds for 10MB file.

4.3 Scalability

Must support at least 10,000 audit records.
Should allow migration to Layer 2 networks.

4.4 Reliability

Smart contract must be immutable once deployed.
Data stored on blockchain must persist permanently.

4.5 Usability

Simple UI
Clear verification results
Responsive design

5 External Interface Requirements

5.1 User Interface

Login via MetaMask
Dashboard
Upload page
Verification page
Audit history page

5.2 Hardware Requirements

Web browser
Internet connection
Ethereum wallet (MetaMask)

5.3 Software Requirements

Node.js
Hardhat
React
Ethers.js
MetaMask

6 System Constraints

Gas fees apply for blockchain transactions.
Blockchain transactions are irreversible.
Requires internet connectivity.
Depends on Ethereum network availability.

7 Assumptions and Dependencies

Users have crypto wallet.

Testnet will be used initially.
SHA-256 is secure for document integrity.

8 Future Enhancements

Multi-signature approvals
IPFS storage integration
Zero-Knowledge Proof verification
DAO-based governance
AI-based risk scoring